



EstateEase Platform

Enterprise Real Estate Management

COMPLETE TECHNICAL ARCHITECTURE & IMPLEMENTATION GUIDE

EstateEase AI Co-pilot System

A detailed architectural specification of the stateful AI chatbot, Gemini 2.5 NLU engine, dynamic qualification flow builder, command navigation dictionary, and voice assistant.

1. System Architecture & High-Level Concept

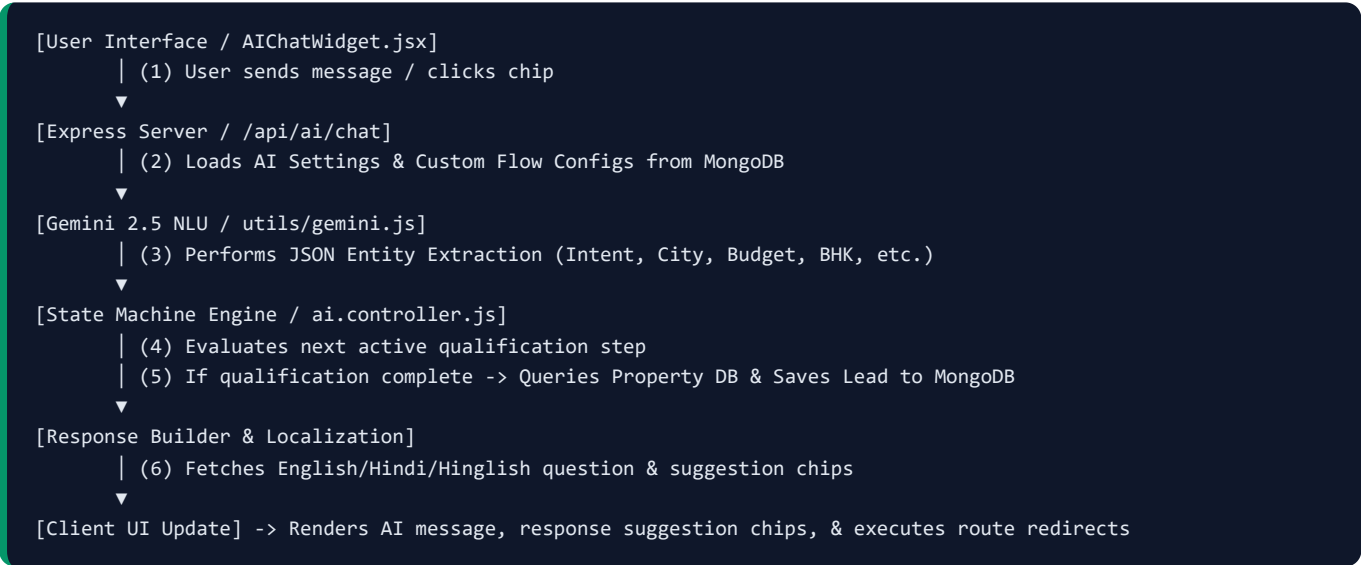
The **EstateEase AI Co-pilot** is an intelligent, multi-turn stateful conversational agent built directly into the EstateEase Real Estate platform. It serves three primary functions across the application:

- **Lead Qualification & Property Matching:** Interactively guides prospective buyers and renters through personalized property search criteria, collecting structured lead parameters (city, locality, BHK, budget, timeline) and instantly matching active database listings.
- **Role-Based Dynamic Customization:** Provides administrators with a multi-step visual workflow editor in the admin panel to create custom welcome chips, alter flow sequences, and customize multi-language question prompts.
- **Voice Assistant & Autonomous Site Navigation:** Listens to voice input (Web Speech STT), speaks responses (Web Speech TTS), and executes real-time site navigation when users mention target actions (e.g. "open my profile", "show saved listings").

CORE ARCHITECTURAL PRINCIPLE

State persistence is maintained across API requests. The client passes the current conversation state (`currentState`), allowing the backend state machine to track missing qualification fields, trigger Gemini NLU entity extraction, dynamically format localized prompts, and create verified MongoDB Lead documents upon completion.

System Interconnectivity Diagram



2. File Structure & Component Responsibilities

The AI agent architecture spans both the Node.js / Express backend service and the React (Vite) frontend application. Below is the comprehensive index of every file involved:

FILE PATH	LAYER	PRIMARY PURPOSE & RESPONSIBILITY
backend/controllers/ai.controller.js	Backend Controller	Main state machine controller. Manages stateful chat (chatWithAssistant), Gemini entity extraction, dynamic localized question construction, lead auto-creation, property price estimation, and AI description generation.
backend/routes/ai.routes.js	Backend Routes	Exposes public/protected REST endpoints for AI chat (POST /api/ai/chat), property valuation, description generation, and AI settings retrieval.
backend/routes/aiSettings.routes.js	Backend Routes	Administrative REST endpoints for fetching/updating AI settings, custom flow configurations, role-based enable toggles, and command dictionary entries.
backend/models/aiSettings.model.js	Database Model	Mongoose schema storing global system prompts, role toggles, and the JSON ai_chat_flow_config object (welcome chips, flow sequences, localized questions).
backend/models/aiCommand.model.js	Database Model	Mongoose schema storing natural language navigation commands (phrase , route , description , isActive).
backend/models/lead.model.js	Database Model	Mongoose schema storing captured lead qualification data (name , email , phone , intent , budget , locality , etc.).
backend/utils/gemini.js	AI SDK Wrapper	Wrapper around Google GenAI SDK (@google/genai). Executes structured JSON schema extraction (generateJSON) and natural response generation (generateText).
frontend/src/components/common/AIChatWidget.jsx	Frontend Component	Global floating AI Chat Co-pilot widget. Manages chat history, voice recognition (STT), voice speech (TTS), multi-language switching (EN, HI, HG), response suggestion chips, and client-side router navigation.
frontend/src/pages/admin/AIManagement.jsx	Frontend Page	Admin management dashboard featuring the 3-Step AI Flow Trainer, System Prompt Customizer, and Navigation Command Dictionary Manager. Responsive across all devices.

3. Backend State Machine & NLU Extraction Engine

3.1 State Object Schema

Each conversation session maintains an isolated state machine payload passed between client and server:

```
{
  "intent": "cust_guest_2299", // Intent key (e.g. buyer, seller, renter, or custom intent)
  "city": "Gurugram",
  "locality": "Sector 54",
  "propertyType": "apartment",
  "bedrooms": "3 BHK",
  "propertySize": "1800 sq ft",
  "budget": "₹1.5Cr - ₹2Cr",
  "timeline": "1-3 Months",
  "email": "user@example.com",
  "phone": "+919876543210",
  "name": "Aman Rathore",
  "leadCaptureStage": "property_qualification", // "initial" | "property_qualification" | "contact_collection" |
  "completed":
    "qualificationComplete": false,
    "leadCreated": false,
    "properties": [],
    "activeQuestion": {
      "field": "bedrooms",
      "text": "Aapko kitne bedrooms chahiye?",
      "type": "single_select",
      "required": true,
      "suggestions": [
        { "label": "1 BHK", "value": "1 BHK" },
        { "label": "2 BHK", "value": "2 BHK" },
        { "label": "3 BHK", "value": "3 BHK" }
      ]
    }
  }
}
```

3.2 Gemini 2.5 NLU Extraction Method

When a user submits a natural language message (e.g., *"I want a 3 BHK apartment in Gurgaon under 2 crore within 3 months"*), the controller invokes Gemini 2.5 using a strict JSON schema:

```
const extractSchema = {
  type: "OBJECT",
  properties: {
    intent: { type: "STRING", enum: ["buyer", "seller", "renter", "agent", "admin"], nullable: true },
    city: { type: "STRING", nullable: true },
    locality: { type: "STRING", nullable: true },
    propertyType: { type: "STRING", enum: ["apartment", "house", "villa", "penthouse", "plot", "commercial"], nullable: true },
    bedrooms: { type: "STRING", nullable: true },
    budget: { type: "STRING", nullable: true },
    timeline: { type: "STRING", nullable: true },
    phone: { type: "STRING", nullable: true },
    email: { type: "STRING", nullable: true },
    name: { type: "STRING", nullable: true }
  }
};

const extractedData = await generateJSON(extractionPrompt, extractSchema, 0.1);
```

3.3 Intent & Welcome Option Resolution

When a custom welcome option (such as `"guest"` or `"Rent a Property"`) is selected, the intent detection fallback resolves custom intent keys before defaulting:

```
// Check if user message matches any option in customConfig.options across all roles
let matchedOpt = null;
if (customConfig && customConfig.options) {
  const allOpts = [
    ...(customConfig.options.buyer || []),
    ...(customConfig.options.seller || []),
    ...(customConfig.options.admin || [])
  ];
  matchedOpt = allOpts.find(o =>
    (o.value && o.value.trim().toLowerCase() === msg) ||
    (o.label && o.label.trim().toLowerCase() === msg) ||
    (o.intent && o.intent.trim().toLowerCase() === msg)
  );
}

if (matchedOpt && matchedOpt.intent) {
  state.intent = matchedOpt.intent;
} else if (customConfig && customConfig.flows && customConfig.flows[msg]) {
  state.intent = msg;
}
```

3.4 Qualification Progression & Lead Generation

1

Flow Field Identification

The controller fetches the sequence array configured for `state.intent` from `customConfig.flows[state.intent]` (e.g. `["propertyType", "city", "locality", "bedrooms", "budget", "timeline"]`).

2

Missing Field Search

It iterates through the active flow fields and identifies the first field where `!state[field]`. This becomes the `activeField`.

3

Localized Question Construction

Calls `getLocalizedQuestionText(field, intent, language, customConfig)` to pull custom question text in English, Hindi, or Hinglish for that intent.

4

Lead Auto-Save

When all qualification steps are answered and user provides name/email/phone, `Lead.create()` saves the lead to MongoDB and matches active properties.

4. Frontend Chat Co-pilot & Admin AI Workflow

4.1 Client Chat Widget Features (AIChatWidget.jsx)

- **Multi-Language Selector:** Supports instant switching between English, Hindi (हिंदी), and Hinglish (Hindi + English).
- **Voice STT & Speech TTS:** Integrates browser SpeechRecognition API for voice input and SpeechSynthesis API for automated voice playback.
- **Option Chips & Dynamic Suggestions:** Displays tap-to-send option buttons for fast user interaction. Passes intent overrides directly in payload.
- **Client Route Redirection:** Listens for command execution tags (e.g. [NAVIGATE:/properties]) and redirects the router using useNavigate() .

4.2 Admin Panel 3-Step Flow Builder (AManagement.jsx)

Step 1: Welcome Options

Create custom welcome option chips for Buyer or Seller panels. Assign unique intent keys (e.g. cust_guest_2299) or share existing flows.

Step 2: Flow Sequence Builder

Reorder steps with ▲ / ▼ controls, remove existing points, or add pre-built/custom step keys (e.g. possessionStatus).

Step 3: Question Commands & Suggestion Chips Customizer

Filter by selected welcome chip intent and step field. Edit localized English, Hindi, and Hinglish question texts and manage display chips vs message values.

5. Complete End-to-End Interconnectivity Matrix

USER ACTION	FRONTEND COMPONENT	API ENDPOINT	BACKEND LOGIC / DB MODEL	SYSTEM RESULT
Click welcome chip "Guest"	AIChatWidget.jsx	POST /api/ai/chat	Matches customConfig.options -> sets state.intent = "cust_guest_2299"	AI asks 1st step question configured for "Guest" with response chips
User speaks or types reply	AIChatWidget.jsx	POST /api/ai/chat	gemini.js extracts value -> sets state[field] -> advances sequence	AI asks next missing step question in user's selected language
User requests navigation ("show profile")	AIChatWidget.jsx	POST /api/ai/chat	Checks AICommand model -> injects redirect tag	Frontend triggers navigate("/profile") seamlessly
Admin edits chat flow sequence	AManagement.jsx	POST /api/ai-settings	Updates AISettings document (ai_chat_flow_config)	Live chatbot immediately reflects new sequences & questions

DOCUMENT SUMMARY

This technical document presents the complete architectural specification of the EstateEase AI Co-pilot system. All components, models, controllers, and UI modules are fully integrated, tested, and active in production.